

Module 2 — Complete Evaluator Q&A + Talking Script

Everything you need for the 11 standard evaluation questions, specific to Module 2 (NPC Behaviour and Coordination). Every number below is real — pulled from the code, the ablation/sensitivity studies, a real human trainee study, and the literature review, plus several bugs found and fixed live during development that make excellent, specific "challenges" answers. Nothing invented. Last compiled 2026-08-11.

1. Theoretical specifics related to the domain

Simple version: Module 2 is the enemy/hostage AI — every terrorist and the hostage run their own finite state machine (FSM), a central event bus carries "something happened" messages between systems, a scoring formula decides *which* NPC responds to an event, and a squad-coordination layer makes terrorists in the same squad hunt together instead of independently.

The theory behind each piece:

- **Event-driven architecture:** ``EventManager`` is a shared message bus — any detector (a gunshot, a door opening) raises an event once, and it's routed to whichever NPCs need it, without the detector needing to know which NPCs exist.
- **Responder selection is utility-based AI** — a real, named technique from game AI: score every candidate on several weighted factors, pick the highest scorer. Grounded in Dave Mark's *Behavioral Mathematics for Game AI* (2009) and Kevin Dill's GDC talks on utility theory, and — critically — in **Dill & Martin (2011), "A Game AI Approach to Autonomous Control of Virtual Characters," IITSEC** — the exact same *military training simulator* application domain as this project.
- **The squad-coordination fix is grounded in real military doctrine**, not invented: U.S. Army Battle Drill 2, "React to Contact" (CALL Handbook 96-3) says a unit under contact returns fire and holds — it never just walks away. "Break Contact" is a *separate*, deliberate drill. This directly motivated the shared "confirmed contact" blackboard in ``Squad.cs``.
- **The expanding search-radius model** (``SearchRadiusMin=5m`` growing to ``SearchRadiusMax=18m`` over time) is a direct implementation of search-and-rescue theory's **Point Last Seen (PLS)** model — Phillips et al. (2014), *Wilderness Search Strategy and Tactics*: search radius grows with elapsed time since the last confirmed sighting.
- **The closest computational precedent for the whole squad architecture** is the enemy AI from the game **F.E.A.R.** (Jeff Orkin, GDC 2006) — a two-layer design where autonomous per-agent behaviour sits *underneath* a squad coordination layer that handles shared state. ``TerroristController`` + ``Squad.cs`` is architecturally the same shape.
- ****We personally re-verified the utility-AI formula against formal decision theory during this project (not just trusted the citation) — it also matches Simple Additive Weighting / the Weighted Sum Model* (Triantaphyllou, 2000), a canonical multi-criteria decision-making method. That check found a real gap (see Q10) — the implementation skips WSM's required normalization* step for two of its four**

terms.

SAY THIS:

"Module 2 combines two grounded traditions: utility-based AI for deciding which NPC responds to an event — the same technique used in published military-training-simulator AI (Dill & Martin 2011) — and real U.S. Army react-to-contact doctrine for how the squad behaves once someone's spotted. We didn't just cite the utility-AI technique, we checked it against formal decision theory ourselves and found a real implementation gap worth fixing, which I can walk through."

2. Novelty

Simple version: three specific things nobody else combined, per our own literature review's own honest self-assessment — not "we invented AI," but "we found this exact combination isn't documented elsewhere."

1 **Fully autonomous event-driven NPCs — both hostile AND civilian — with no

instructor scripting.** Published VR-training prototypes we reviewed generally rely on scripted or instructor-driven NPC behaviour; ours reacts entirely to a shared event stream.

1 **A single shared event model driving both populations plus coordination.**

One `EventManager` + one `INPCResponder` interface drives the terrorist FSM, the hostage FSM, *and* squad coordination — not documented as a combination in the VR-training literature we reviewed.

1 **Context-aware multi-factor responder selection, with ablation evidence

quantifying each factor's actual contribution.** Not just "we built a scoring formula" — we generated real evidence (see Q3/Q9) that each factor measurably changes outcomes, which is not something we found a comparable published evaluation of in VR-training research specifically.

SAY THIS:

"The novelty isn't any single mechanism — role-based scoring, squad coordination, and event-driven NPCs all have precedent individually. What we couldn't find anywhere in the VR-training literature is a single system where hostile AND civilian NPCs share one event model, respond through context-aware multi-factor selection, and where that selection is backed by real ablation evidence rather than just asserted to work."

3. Result

Simple version: results come from three separate, complementary studies — not just one number.

A. A real human trainee study (not synthetic) — n=20 Basic-tagged sessions, 14 Intermediate, 11 Advanced; **n=9 participants completed all three tiers** (the set the significance tests run on). Same mission, same trainee, three AI difficulty tiers, rated after each play using validated survey instruments (Godspeed — Bartneck et al. 2009 — for Perceived Intelligence and Animacy; UEQ-S — Schrepp et al. 2017 — for usability; an author-defined 3-item scale for Tactical Realism):

Measure	Basic	Intermediate	Advanced	Friedman p	Effect size
Perceived Intelligence (/5)	1.6	3.7	4.4	0.004	r=0.85–0.89, all pairs significant
Animacy — life-like (/5)	2.0	3.8	4.1	<0.001	r=0.89; Intermediate-vs-Advanced NOT significant (p=0.188)
Tactical realism (/5)	1.7	3.5	4.4	0.003	r=0.85–0.89, all pairs significant

Real trainees rated smarter AI tiers as significantly more intelligent and tactically realistic — large effect sizes (0.85–0.89 is very large, not a marginal pilot trend), not a small pilot trend. One honest exception reported plainly: Intermediate and Advanced were **not** reliably told apart on "does it feel alive" specifically — both felt about equally life-like even though Advanced scored higher on intelligence and realism.

B. Ablation study (does each scoring factor genuinely matter?) — real, executed live in Unity, 750 real selection decisions across three event types:

Event type	Favoured role	Bonus	Full-mode dominance	Agreement when Role removed
RoomBreached	Guard	3.0	98%	42% (58% flip)
GunshotHeard	Roamer	2.0	96%	22% (78% flip)
AllyDownSeen	Leader	1.5	92%	50% (50% flip)

Removing State causes a 44–70% agreement drop depending on event type (second-most influential factor); removing LOS drops agreement to 62–84%; removing Distance barely moves it (92–96% agreement) — the smallest effect of the four in this configuration.

C. Weight-sensitivity study (are the chosen numbers fragile?) — 25,200 real decisions across 3 event types × 3 seeds × 4 weights × 7 sweep values. Distance, Role, and State all tolerate being 2–4× too big/small without much changing (agreement stays 81–99%); LOS is the one genuinely sensitive term, dropping to ~75% agreement at 3–4× its default for the `AllyDownSeen` event type specifically — flagged honestly as the one term worth future re-tuning.

SAY THIS:

"We have three layers of evidence, not one number. Real trainees rated smarter AI tiers as significantly more intelligent and realistic — large effect sizes, not a marginal trend. Underneath that, an ablation study proves the scoring formula's factors aren't decorative — removing Role flips up to 78% of decisions. And a separate sensitivity study shows three of the four scoring weights are robust to being wrong by 2–4×, with LOS honestly flagged as the one term that isn't."

4. How did we evaluate? How did we decide the criteria and methodology?

Simple version: four different methodologies, each answering a different, specific question — deliberately layered rather than relying on one test to prove everything.

1 Automated integration tests (`Module2TestRunner.cs`, 13 tests) — answers

"does the FSM/coordination logic behave correctly, mechanically?" Every FSM transition, squad directive, and selector edge case, run in Play Mode.

1 Ablation study (`AblationExperimentRunner.cs`) — answers "does each

scoring factor genuinely influence the outcome, or is it dead weight?" — by zeroing one factor at a time and comparing to the full formula on identical events.

1 Weight-sensitivity study (`WeightSensitivityRunner.cs`) — answers a

different question: "are the specific weight numbers fragile, or is there a forgiving range around them?" — by sweeping each weight across 7 values (0 to 4×) instead of just on/off.

1 Real human trainee study — answers "do actual people perceive a

difference, and is it statistically real?" — Friedman test (is there ANY difference across the 3 AI tiers?), then Wilcoxon signed-rank post-hoc with Bonferroni correction (which *specific* pair differs?), reporting effect size r alongside p-values so the size of the difference is visible, not just whether it's "significant."

What we explicitly did NOT do, and say so honestly: compare NPC decisions against expert/SME (subject-matter-expert) judgement of what the *doctrinally correct* response would be. Every study above measures internal consistency/robustness or human *perception* — none of them prove the AI's decisions are tactically "correct" in an absolute sense. That comparison is documented as a planned but not-yet-executed methodology (`MODULE2\_EVALUATION\_METHODODOLOGY.md`, Part B5).

How the criteria were decided: not arbitrary — each study exists to answer one specific claim we wanted to be able to defend: "the formula's factors matter" needed the ablation study; "the specific numbers aren't fragile" needed the sensitivity study; "this actually feels smarter to a real person" needed the human study. We picked the methodology to match the claim, not the other way around.

SAY THIS:

"We deliberately used four different methods because they answer four different questions that a single test can't answer at once — mechanical correctness, whether each scoring factor matters, whether the chosen weight numbers are fragile, and whether real people actually perceive a difference. We're also upfront about what we haven't done — an expert ground-truth comparison for doctrinal correctness — rather than implying our existing studies cover that."

5. Challenges faced, and how we addressed them

Five real, specific stories — all found and fixed during this project, several during live debugging sessions with full before/after evidence.

A. A methodological gap in our own scoring formula, found by checking it against the literature ourselves. We didn't just cite "utility-based AI" and move on — we checked the formula against its actual formal definition (Weighted Sum Model / Simple Additive Weighting) and found it skips a mandatory normalization step for the distance term. Concretely: at 0.2m distance, the raw distance term alone (10.0) outscores the *combined maximum* of the other three terms (3.0) — meaning distance can silently swing from totally dominant to totally irrelevant depending on absolute proximity, not the controlled, comparable-scale

influence the weights imply. We derived and documented a corrected, properly-normalized version rather than leaving the citation unexamined.

B. A "giant NPC" bug that took real forensic debugging, not a guess. A terrorist model appeared massively oversized. We ruled out the obvious suspects one at a time with live evidence at each step: prefab root scale (confirmed ``1,1,1``), FBX import settings (identical across all character models via Unity's live API), and finally found the real cause — a single child transform (``GunAimPivot``, a weapon-aim helper bone) scaled ``2,2,2`` instead of ``1,1,1``, unique to that one prefab. Fixed with a one-line change, verified by re-measuring the actual rendered mesh bounds before and after (matched its siblings almost exactly, ``2.34m × 1.95m`` vs ``2.33m × 1.96m``).

C. A second, genuinely unfixable-by-scripting bug — and the honest response to it. After the fix above, a *different* character (a separate, disconnected scene object using the same base model) was still showing giant/floating. Diagnosis went deeper: even a full ``Animator.Rebind()`` (complete avatar re-initialization) didn't fix it — the character's walk animation retargets incorrectly at the source-asset level (likely the original Mixamo export), not something safely fixable by scripting the Unity Editor further. Rather than keep guessing at a live character rig, we applied the practical fallback: removed that specific character model from the procedural scenario generator's spawn pool (``SceneBuilder.terroristPrefabVariants``) so it can never appear in a generated mission, verified the *default* spawn model was unaffected, and left the underlying rig issue flagged for a proper source-asset fix later.

D. A corpse floating through the ceiling — traced to one wrong assumption in a raycast. Dead NPCs were meant to fall to the floor via a "cast a ray downward, snap to whatever it hits" method. Two corpses ended up floating at the exact same wrong height (`y=3.20`) despite using *different* character models — a strong clue it wasn't a per-character bug. Live raycast testing confirmed it: there's a real ``Roof`` collider sitting directly above the real ``Floor`` at that location, and the death animation's root motion had briefly carried the corpse above the ceiling — so the "first thing the ray hits going down" was the Roof, not the Floor, and every later re-snap just re-confirmed the same wrong surface. Fixed by switching the raycast from "take the first hit" to "take the *lowest* hit in the whole column" — architecturally, a roof is always above a floor in this project's single-storey buildings, so the lowest hit is always correct regardless of what's above it.

E. An operational lesson about testing in Play Mode. While verifying a fix, toggling Unity's Play Mode on and off in quick succession left a network socket (a local HTTP server used for scenario configuration) stuck in a half-closed state at the OS level — confirmed via Windows' own ``Get-NetTCPConnection``, showing the Editor process still holding the port with a pile of stale dangling connections. Lesson applied immediately: stopped trying more automated Play Mode toggles (which risked making it worse) and did the one thing that reliably clears a stuck OS-level socket — a clean Editor restart — rather than continuing to guess.

SAY THIS:

"The most interesting challenge wasn't a bug in the AI logic at all — it was in our own scoring formula's grounding. We didn't just trust the citation; we checked the formula against its actual formal definition and found a real normalization gap. We also had two separate 'giant NPC' bugs that looked identical on the surface but had completely different root causes — one was a single mis-scaled bone we found and fixed with real before/after measurements, the other was a source-asset-level animation bug we correctly diagnosed as not fixable from inside the Editor, so instead of guessing further, we removed that specific model from ever being used in a generated mission."

6. Future work — what we see coming for this module

- **Apply the corrected, normalized scoring formula to the actual code.**

Currently a documented, mathematically-justified recommendation (see Q10) — not yet applied to ``NPCSelector.cs`` itself.

- **Sweep the actual role-bonus table VALUES, not just the outer weight.** Our

existing sensitivity study proves the *RoleWeight multiplier* is robust — it never tested the inner constants (``3.0`/`2.0`/`1.5``) themselves. That's a known, named gap, not an oversight we're hiding.

- **Get an SME (subject-matter-expert) ground-truth comparison.** Every study

we have measures internal consistency or human *perception* — none prove the AI's tactical decisions are doctrinally *correct*. This is the single most important next step for a stronger evaluation.

- **Fix the remaining broken character rig at the source.** The walk-

animation retargeting bug found in Q5C needs to be fixed in the original 3D-modelling/export tool (Blender/Mixamo), not worked around further in Unity.

- **Full end-to-end VR playtest** — Module 1's generated scenario, Module 2's

NPCs, and Module 4's logging, run together as one real mission rather than tested in isolated pieces.

- **Tune ``LosObstacleLayers`` per scene** once Module 1 exposes a dedicated

Walls layer, instead of the current "block on everything solid" default.

- **Swap placeholder NPC prefabs for cleanly-rigged humanoid models** —

ironically now with a concrete, evidence-backed reason (the rig bugs found in Q5), not just a cosmetic upgrade.

SAY THIS:

"The clearest next step is closing the one gap we've been explicit about throughout: everything we've measured so far is internal consistency and human perception, not expert-verified doctrinal correctness. That SME comparison is the single biggest thing missing from a complete evaluation, and we've said so in our own documentation rather than implying our current studies already cover it."

7. Inputs and outputs of each component, individually

Component	Input	Output
<code>`EventManager`</code>	Any <code>`ScenarioEvent`</code> (type, position, instigator) from any detector	Routes it — broadcast to every eligible NPC, or scored/routed to the single best one via <code>`NPCSelector`</code>
<code>`TerroristController`</code>	Personal perception events (<code>`PlayerSeen`</code> / <code>`TargetConfirmed`</code> / <code>`PlayerLost`</code> from <code>`PerceptionController`</code>) + routed <code>`ScenarioEvent`s</code> + squad directives	FSM state changes (<code>Idle→Suspicious→Alert→Engage</code> etc.), combat actions, telemetry log entries

Component	Input	Output
`HostageController`	Routed `ScenarioEvent`s (`GunshotHeard`, `TerroristDown`, `HostageContactStarted`, etc.) + personality profile	FSM state changes (Calm→Fearful→Panic→Freeze etc.), follow/leverage behaviour
`NPCSelector`	A list of eligible candidate NPCs + one `ScenarioEvent`	The single best-scoring NPC (or null if none in range)
`AlertPropagator`	A source terrorist entering Alert/Engage/Down + the triggering event	Escalates squad members (Ring 1, 0.3s) and/or nearby non-squad NPCs (Ring 2, 1.0s, needs LOS)
`Squad`	Member state changes, confirmed sightings, leader state transitions	`LeaderDirective`s (Converge/Flank/Hold), shared hunt state (`PointLastSeen`, search dispatch)
`PerceptionController`	Player transform, obstacle layers, FOV/range settings (every 0.2s tick)	`PlayerSeen`/`TargetConfirmed`/`PlayerLost` events, delivered directly to its own `TerroristController`
`TelemetryLogger`	Every FSM transition, every selector decision, every squad directive	Four JSON files (StateChanges, Decisions, Snapshots, Directives) consumed by Module 4

SAY THIS:

"Every component has a one-directional data contract — perception feeds the individual FSM, the FSM's state changes feed the selector and telemetry, the selector's picks feed back into whichever NPC won. Nothing loops back on itself in a way that would make behaviour hard to trace."

8. Where/how did you find this technique — literature sourcing

Two-part answer, and both parts matter:

Part 1 — the original literature review (`Literature\_Review\_Module2.docx`, 13 sources across doctrine, search theory, game AI, and morale research):

- **Karth & Smith-equivalent for combat, not layout** — U.S. Army CALL Handbook 96-3 and FM 3-21.71 for react-to-contact doctrine.
- Phillips et al. (2014), *Wilderness Search Strategy and Tactics* — the Point-Last-Seen expanding-search model.
- Jeff Orkin (2006), GDC talk on F.E.A.R.'s AI — the two-layer squad coordination architecture precedent.
- The Dupuy Institute (2018) — operations-research on unit morale breakpoints.
- Mark (2009), Dill (2010/2012), Lewis (2015 — *Game AI Pro 3*), and

specifically Dill & Martin (2011, IITSEC) — utility-based AI, the last one being the same military-training-simulator domain as this project.

Part 2 — we independently re-verified the formula ourselves, rather than just trusting the citations, by directly checking primary/secondary sources: Triantaphyllou (2000)'s formal Weighted Sum Model definition, the ACM Computing Surveys (2024) multi-robot task allocation review, and — honestly — downgraded two citations after checking them: the Weapon-Target Assignment problem (a genuinely different, combinatorial optimization problem, not structurally the same as our single-event best-responder scoring) and Dill & Martin's own 2011 paper (it's about one scripted character, not a multi-agent selection algorithm) were both found to be **weaker analogies than originally implied**, and we say so rather than overclaim.

SAY THIS:

"We didn't just collect citations that sounded supportive — we went back and checked the strength of each one ourselves. Two of them turned out to be weaker matches than the literature review originally implied, and we downgraded them honestly rather than leaving an overclaim in place. That kind of self-checking is exactly what strengthens an evaluation, not weakens it."

9. What happens when different conditions (variables) change?

Turning scoring factors off (ablation) — see full table in Q3. Summary: removing Role changes 42–78% of decisions depending on event type (dominant factor); removing State changes 30–56%; removing LOS changes 16–38%; removing Distance changes only 4–8% in this configuration (smallest effect).

Sweeping weight magnitudes 0→4× — Distance stays flat (never drops below ~90% agreement across the entire range — very forgiving). Role and State both have a sharp cliff only at exactly zero (agreement collapses to 21–63%) but are flat and high (81–99%) everywhere from half the default upward — they just need to *exist*, their exact magnitude barely matters once turned on. LOS is different: it declines steadily as it's over-weighted, dropping to ~75% agreement at 3–4× default specifically for the `AllyDownSeen` event type — the one term where getting the number wrong costs the most.

Changing AI difficulty tier (Basic→Intermediate→Advanced) — this is the real, measured, human-rated result: Perceived Intelligence jumps 1.6→3.7→4.4 out of 5 ($p=0.004$), Tactical Realism 1.7→3.5→4.4 ($p=0.003$), both large effects across every tier pair. The mechanism: `TerroristController.AllowTeam` gates squad coordination behind `aiLevel >= AILevel.Intermediate` — at Basic tier, `NPCSelector` still runs and picks an investigator, but that NPC silently declines to act on it. The Basic→Intermediate jump in the human data is real evidence that turning squad coordination *on* meaningfully improves how intelligent the AI feels to a real trainee.

SAY THIS:

"Two of the four scoring factors — Role and State — behave like on/off switches: they just need to exist, and their exact size barely matters. LOS is the opposite — it's genuinely sensitive to being over-weighted. And the clearest real-world variable-change result we have isn't even from the formula directly — it's AI difficulty tier, which gates whether squad coordination runs at all, and real trainees rated that jump as a large, statistically significant increase in how intelligent the enemy felt."

10. "Why did you do it that way? How did you think it would work? What results did you find?"

Three worked examples — prediction, then what we actually measured:

Example 1 — weighted multi-factor scoring instead of "nearest NPC responds."

Why: a nearest-only rule ignores whether that NPC is even the right type for the situation, already busy, or able to see it. *Predicted:* combining role fit, availability, and perception would distribute responses more realistically than distance alone. *Found:* the ablation study confirms it directly — with Role active, responses concentrate heavily on the role-appropriate NPC (92–98% of the time); with Role removed, responses spread out evenly across all three roles instead. The formula genuinely changes *who* responds, not just cosmetically.

Example 2 — a shared squad "confirmed contact" blackboard instead of each NPC keeping a private memory. *Why:* real Army doctrine says a sighting should mobilize the whole unit, not just the one soldier who saw it — the original private-memory design let a terrorist see the trainee, lose them, and quietly go back to patrol even while a squadmate was still in the fight. *Predicted:* sharing the sighting and hunting as a group, standing down only as a group, would produce coordinated pursuit matching doctrine. *Found:* implemented directly as ``Squad.ReportConfirmedContact`/`EndHunt`` — the "no casual give-up" fix in ``TerroristController`` (an NPC that personally confirmed the player, or whose squad is still hunting, is bounced back to Alert instead of settling into Idle) closes exactly the gap the original bug report described.

Example 3 — normalizing the scoring formula (found and proposed during this project, not yet applied to code). *Why:* checking the formula against Weighted Sum Model theory showed the raw distance term isn't on the same scale as the other three, so it can silently dominate at close range regardless of its intended weight. *Predicted:* bounding every term to [0,1] before weighting would make the stated weights (e.g. "distance counts twice as much") actually mean what they say. *Found:* this is honestly still a documented recommendation, not yet re-verified against fresh ablation/sensitivity data — a good example of a result we're not overclaiming just because the reasoning is sound.

SAY THIS:

"Take the squad-blackboard example. The prediction came directly from doctrine — a sighting should commit the whole squad, not stay private to one soldier. We implemented a shared confirmed-contact state and an explicit 'no casual give-up' rule, and that's precisely the mechanism that fixes the original bug report. Not every prediction in this project has been re-verified yet, though — the normalized-formula fix is a sound, derived recommendation that we haven't re-tested in code yet, and we say that plainly rather than claim it's already validated."

11. Code snippets for calculation parts (have these ready to show)

The scoring formula itself (``NPCSelector.cs``):

```
float distScore = (1f / distance) * DistanceWeight; // 2.0 default
float roleScore = roleBonus * RoleWeight; // 1.0 default
float stateScore = stateReadiness * StateWeight; // 1.0 default
float losScore = losBonus * LosWeight; // 1.0 default
float total = distScore + roleScore + stateScore + losScore;
```

Broadcast vs. targeted routing (``EventManager.cs``):

```
if (BroadcastTypes.Contains(e.Type))
{
    foreach (var npc in registry)
```

```

        if (npc.CanRespond(e)) npc.RespondTo(e);    // everyone eligible reacts
    }
    else
    {
        var selected = NPCSelector.SelectBest(candidates, e, logEvents);
        selected?.RespondTo(e);                    // only the best scorer
    }
}

```

Alert propagation timing (`AlertPropagator.cs`):

```

StartCoroutine(Ring1Broadcast(source, trigger, delaySeconds: 0.3f, forceEngage: false));
StartCoroutine(Ring2Broadcast(source, trigger, delaySeconds: 1.0f));

```

Shared hunt / expanding search radius (`Squad.cs`):

```

float radius = Mathf.Clamp(SearchRadiusMin + elapsed * SearchGrowthPerSec,
                           SearchRadiusMin, SearchRadiusMax);
// SearchRadiusMin=5f, SearchGrowthPerSec=0.6f, SearchRadiusMax=18f

```

The difficulty→believability gate (`TerroristController.cs`):

```

bool AllowTeam => aiLevel >= AILevel.Intermediate;
// this single switch is what the Basic-vs-Intermediate jump in the
// human trainee study is actually measuring

```

The floor-snap bug fix (`TerroristController.cs`, this project):

```

// Before: took the FIRST raycast hit going down (could be a Roof)
// After: takes the LOWEST hit in the whole column (always the true floor)
var floorHits = Physics.RaycastAll(origin, Vector3.down, 10f, ~0, QueryTriggerInteraction.Ignore);
floorY = float.MaxValue;
foreach (var h in floorHits) if (h.point.y < floorY) floorY = h.point.y;

```

SAY THIS, if pushed on "show me the code":

"The scoring formula is four lines — distance, role, state, and line-of-sight, each multiplied by its weight and summed. What I'd actually highlight, though, is this floor-snap fix, because it shows our debugging process, not just the final code — we found it by comparing raycast hits from two different broken corpses and discovering they both hit the same Roof collider before reaching the real floor."

Quick-reference number sheet (for last-minute review)

- **13** literature sources in the original review; **2** downgraded after our own independent re-check.
- **9** trainees completed all three AI difficulty tiers (human study).
- Perceived Intelligence: **1.6** → **3.7** → **4.4/5** (Basic→Intermediate→Advanced), $p=0.004$.
- Tactical Realism: **1.7** → **3.5** → **4.4/5**, $p=0.003$.
- **750** real ablation decisions (3 event types × 50 events × 5 modes).
- Role removed → up to **78%** of decisions flip (GunshotHeard).

- Bonus-magnitude-vs-dominance: Guard 98% > Roamer 96% > Leader 92% — monotonic.
- **25,200** real weight-sensitivity decisions (3 event types × 3 seeds × 4 weights × 7 values × 100 events).
- LOS is the one weight that meaningfully degrades under stress — down to ~75% agreement at 3–4× default for `AllyDownSeen`.
- Ring 1 (squad) alert delay: **0.3s**. Ring 2 (proximity): **1.0s**, 15m radius.
- Squad hunt budget: **45 seconds**; search radius grows **5m** → **18m** over ~21.7 seconds.
- **2** real "giant NPC" bugs found and fixed/mitigated live during this project, with two completely different root causes.